

Image Inpainting and Object Removal

A comparison of classical, CNN, Transformer, and diffusion-based approaches

Ivan | Aleksandr | Egor | Tatiana | Computer Vision 2026 | October 6, 2026

Abstract

This project develops an image inpainting application for removing user-selected objects or regions from photographs and reconstructing the missing content. Four approaches will be compared: OpenCV Telea as a classical baseline, LaMa as a CNN-based method, ViT-Inpainter as a Transformer-based method, and FLUX.1-Fill as a diffusion-based method. The project is deliberately separated into two modes. The **Evaluation Mode** is a reproducible console benchmark based on the CORNE-Val paired object-removal dataset and is used to compare reconstruction quality and inference speed. The **Application Mode** is an interactive web application that accepts arbitrary user images and masks and displays the four results in a progressive carousel. No model is trained or fine-tuned from scratch; all methods are evaluated using pretrained weights.

Contents

1	Project Scope	1
2	Dataset Preparation	2
2.1	Dataset split	3
2.2	Mask conditions	3
3	Experimental Plan	3
3.1	Evaluation metrics	3
3.2	Sensitivity experiment	4
4	Implementation Plan	4
4.1	Evaluation Mode	4
4.2	Application Mode	4
4.3	Main resources	5
5	Team Responsibilities	6
6	Conclusion	6

1 Project Scope

The project addresses the problem of **masked object and region removal**. Given an RGB image and a binary mask specifying the area to remove, the system must reconstruct a visually plausible

image in which the masked content has been replaced by an appropriate background.

The project does not include object detection or automatic mask generation as its primary research problem. The mask is treated as an input to the inpainting system. In Application Mode, the user creates the mask manually using the web interface.

Research question

How do classical, CNN-based, Transformer-based, and diffusion-based inpainting methods compare in reconstruction quality and inference speed under different mask conditions?

The experimental comparison consists of four approaches:

- **OpenCV Telea** – classical inpainting baseline;
- **LaMa** – CNN-based image inpainting;
- **ViTInpainter** – Transformer-based image inpainting;
- **FLUX.1-Fill** – diffusion-based generative inpainting.

All methods receive the same image and the same mask for a given evaluation sample. No model is trained from scratch and no model is fine-tuned during the main experiment.

2 Dataset Preparation

The primary evaluation dataset will be **CORNE-Val**, a paired object-removal benchmark containing object-present photographs, corresponding object-free backgrounds, and two types of masks.

The dataset provides the following logical structure:

- **shot/** – image containing the object to be removed;
- **bg/** – corresponding object-free background;
- **mask_sam/** – object-core mask;
- **mask_eff/** – effect-aware mask covering the object and associated visual effects.

For each sample, the evaluation input is formed from:

$$(\text{shot}, \text{mask})$$

while the paired clean background serves as the reconstruction target:

$$\text{prediction} \rightarrow \text{bg}.$$

This provides a direct ground truth for quantitative evaluation and avoids treating an image containing the original object as the desired output.

2.1 Dataset split

No additional train/validation/test split will be created for the main experiment because the project uses pretrained models and performs no model training or fine-tuning.

CORNE-Val will instead be treated as a fixed **held-out evaluation benchmark**. The same samples, preprocessing settings, and evaluation protocol will be used for all four approaches.

2.2 Mask conditions

Two native mask conditions will be evaluated:

- **Object-core mask** – removes the main object region;
- **Effect-aware mask** – covers the object and associated visual effects such as shadows or other visible remnants.

Additional derived masks may be used for the sensitivity experiment, including controlled dilation and simplified geometric masks such as bounding boxes.

3 Experimental Plan

The experiment compares the four approaches under identical input conditions.

Approach	Type	Role
OpenCV Telea	Classical	Baseline
LaMa	CNN	Learned inpainting
ViTInpainter	Transformer	Transformer-based comparison
FLUX.1-Fill	Diffusion	Generative comparison

Table 1: Approaches included in the main experiment.

3.1 Evaluation metrics

The main quantitative measures are:

- **PSNR** $\uparrow$ – pixel-level reconstruction similarity;
- **SSIM** $\uparrow$ – structural similarity;
- **LPIPS** $\downarrow$ – perceptual similarity;
- **Inference time** $\downarrow$ – computational efficiency.

The reconstruction metrics will be evaluated with respect to the object-free ground-truth background. Where appropriate, metrics will focus on the masked or affected region rather than only evaluating unchanged pixels outside the edit area.

3.2 Sensitivity experiment

The required sensitivity experiment will investigate the effect of mask construction on reconstruction quality.

The planned conditions are:

Condition	Description
Object-core	Native object mask
Effect-aware	Object + associated visual effects
Dilated	Expanded object-core mask
Bounding box	Simplified rectangular approximation

The purpose of this experiment is to determine how sensitive each model is to the precision and extent of the user-provided mask.

4 Implementation Plan

The system will be implemented in Python 3.12 using a shared inference interface for all four methods. The application and evaluation modes will use the same model implementations to ensure consistency between the benchmark and the final demo.

4.1 Evaluation Mode

Evaluation Mode will be a console application used exclusively for benchmarking.

A typical workflow will be:

1. Load the fixed CORNE-Val samples.
2. Load the corresponding object mask.
3. Run OpenCV Telea, LaMa, ViTInpainter, and FLUX.1-Fill.
4. Save the generated results.
5. Calculate PSNR, SSIM, LPIPS, and inference time.
6. Aggregate the results into a comparison table.
7. Save representative examples and failure cases.

The output will contain both per-image results and aggregated statistics for the final report.

4.2 Application Mode

Application Mode will provide the interactive user-facing system.

The intended workflow is:

1. Upload an arbitrary photograph.
2. Draw or select a mask over the region to remove.
3. Submit the image and mask for processing.

4. Run the four inpainting approaches.
5. Display their outputs in a horizontal carousel.

Each carousel position represents one model. A result is displayed as soon as it becomes available. While a model is still processing, its carousel item displays a loading placeholder rather than blocking the other completed results.

Application behavior

The application does not calculate PSNR, SSIM, or LPIPS for arbitrary user images because no ground-truth object-free image is available. Application Mode is therefore intended for qualitative inspection, while Evaluation Mode is responsible for quantitative comparison.

4.3 Main resources

The implementation will use:

- Python 3.12;
- PyTorch and torchvision;
- OpenCV;
- Hugging Face Transformers / Diffusers where applicable;
- LPIPS implementation for perceptual evaluation;
- pretrained weights for LaMa, ViTInpainter, and FLUX.1-Fill;
- a backend API connecting the model inference pipeline to the web application.

5 Team Responsibilities

The project is divided into three ML responsibilities and one frontend/backend responsibility.

Member	Area	Responsibilities
Ivan	ML / Team Lead	LaMa implementation; OpenCV baseline; shared inference interface; evaluation pipeline; experiment coordination; final result aggregation.
Aleksandr	ML	ViTInpainter implementation and inference; preprocessing and postprocessing; Transformer-specific experiments; runtime and quality evaluation.
Egor	ML	FLUX.1-Fill implementation and inference; diffusion configuration; memory and runtime investigation; analysis of generative and semantic failure cases.
Tatiana	Frontend / Backend	Implementation of the web application and backend integration; image upload and mask interaction; carousel-based result presentation; connection of the interface to the shared inference API.

Table 2: Division of responsibilities within the team.

All team members will be familiar with the common experimental setup, the evaluation metrics, and the main conclusions of the comparison.

6 Conclusion

The project will combine a controlled quantitative benchmark with a practical interactive application. CORNE-Val provides paired object-removal examples for reproducible evaluation, while the application allows qualitative testing on arbitrary user photographs.

The central comparison will examine whether increasingly sophisticated approaches provide measurable improvements over a classical baseline, and whether those improvements justify their additional computational cost.

A reproducible console benchmark for quantitative evaluation and a web application that allows users to upload an image, draw a mask, and progressively compare OpenCV, LaMa, ViTInpainter, and FLUX.1-Fill results in a carousel.